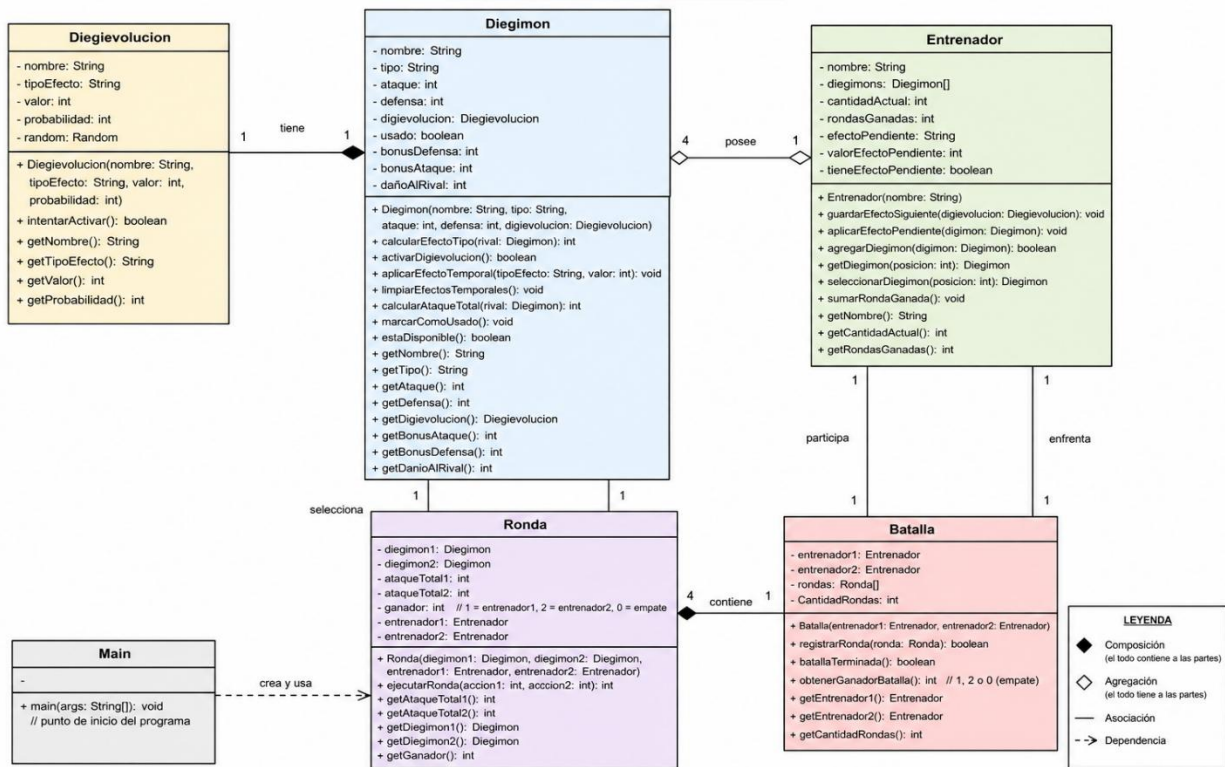


DIAGRAMA UML – BATALLA DE DIGIMONS



Requisitos funcionales del sistema:

1. El sistema debe permitir ingresar el nombre de dos entrenadores.
2. Cada entrenador debe contar con cuatro Digimon diferentes.
3. Cada Digimon debe poseer nombre, tipo, ataque, defensa y una Digievolucion.
4. Cada Digievolucion debe poseer nombre, tipo de efecto, valor de efecto y probabilidad de activación.
5. La batalla debe desarrollarse durante cuatro rondas.
6. En cada ronda, cada entrenador debe selección un Digimon que no haya utilizado anteriormente.
7. El sistema debe impedir que un entrenador seleccione nuevamente un Digimon ya utilizado
8. Cada entrenador debe poder decidir entre atacar o intentar utilizar la Digievolucion.
9. El sistema debe calcular el efecto de tipo entre los Digimon efectivo +20, débil -10 o neutral 0.
10. La Digevolucion debe activarse dependiendo de su probabilidad.
11. Si la Digievolucion se activa, su efecto debe aplicarse en la ronda actual y conservarse para la siguiente ronda.
12. El sistema debe calcular el ataque total de ambos Digimon.
13. El Digimon con mayor ataque total debe ganar la ronda.
14. Si ambos tienen el mismo ataque total, la ronda debe terminar en empate.
15. El sistema debe registrar las rondas ganadas por cada entrenador.
16. Después de cuatro rondas, el sistema debe determinar al ganador de la batalla o indicar que hubo empate.

2. Propósito de cada clase:

1. Digievolucion

Esta clase representa la habilidad especial que tiene cada Digimon. Guarda el nombre de la habilidad, el tipo de efecto que hace, cuánto vale ese efecto y la probabilidad de que se active. También se encarga de decidir de forma aleatoria si la habilidad se activa o no.

2. Digimon

Esta clase representa a cada Digimon de la batalla. Guarda datos como su nombre, tipo, ataque, defensa y Digievolución. También tiene los métodos necesarios para calcular ventajas de tipo, activar habilidades, calcular su ataque total y saber si ya fue usado.

3. Entrenador

Esta clase representa a cada jugador. Guarda su nombre, sus cuatro Digimon y las rondas que ha ganado. También sirve para controlar qué Digimon todavía se pueden usar y guardar los efectos de una Digievolución para la siguiente ronda.

4. Ronda

Esta clase controla lo que pasa en una ronda. Recibe los dos Digimon y los dos entrenadores, procesa las acciones elegidas, calcula los ataques y decide quién ganó la ronda.

5. Batalla

Esta clase se encarga de controlar la batalla completa. Guarda a los dos entrenadores y las rondas jugadas. También cuenta las victorias y al final determina quién ganó la batalla.

6. Main

Es la clase principal del programa. Aquí se crean los entrenadores, Digimon, Digievoluciones y la batalla. También se usa Scanner para recibir las opciones de los jugadores y se muestran todos los mensajes y resultados.

3. Propósito de cada atributo de cada clase:

Clase Diegievolucion

- **nombre: String**
Guarda el nombre de la Digievolución o habilidad especial asociada al Digimon.
 - **tipoEfecto: String**
Indica qué efecto produce la Digievolución. En el programa se utiliza para distinguir entre aumento de ataque, aumento de defensa o daño directo.
 - **valor: int**
Guarda la cantidad numérica que tendrá el efecto de la Digievolución. Por ejemplo, un aumento de 15 puntos de ataque.
 - **probabilidad: int**
Representa el porcentaje de probabilidad que tiene la Digievolución de activarse.
 - **random: Random**
Se utiliza para generar un número aleatorio y determinar si la Digievolución se activa o no según su probabilidad.
-

Clase Digimon

- **nombre: String**
Almacena el nombre del Digimon y permite identificarlo durante la selección y los enfrentamientos.
- **tipo: String**
Guarda el elemento del Digimon, como Fuego, Agua, Planta o Eléctrico. Este atributo se utiliza para calcular ventajas y desventajas de tipo.
- **ataque: int**
Representa el valor base de ataque del Digimon y forma parte del cálculo del ataque total.
- **defensa: int**
Representa la capacidad defensiva del Digimon y se resta del ataque total de su rival.
- **diegievolucion: Diegievolucion**
Guarda el objeto Diegievolucion perteneciente al Digimon, permitiéndole acceder al tipo de efecto, valor y probabilidad de activación.
- **usado: boolean**
Indica si el Digimon ya fue utilizado en una ronda. Sirve para impedir que se seleccione nuevamente.

- **bonusDefensa: int**
Guarda temporalmente los puntos adicionales de defensa obtenidos mediante una Digievolución.
 - **bonusAtaque: int**
Guarda temporalmente los puntos adicionales de ataque obtenidos mediante una Digievolución.
 - **dañoAlRival: int**
Almacena el valor de daño directo provocado mediante una Digievolución para que sea considerado al calcular el ataque total del rival.
-

Clase Entrenador

- **nombre: String**
Guarda el nombre del entrenador para identificar al jugador durante la batalla.
 - **diegimons: Diegimon[]**
Es un arreglo que almacena los cuatro Digimon pertenecientes al entrenador.
 - **cantidadActual: int**
Lleva el control de cuántos Digimon han sido agregados al arreglo. También ayuda a evitar que se agreguen más de cuatro.
 - **rondasGanadas: int**
Guarda la cantidad de rondas ganadas por el entrenador. Al finalizar la batalla se utiliza para determinar al ganador.
 - **efectoPendiente: String**
Guarda el tipo de efecto de una Digievolución que debe continuar aplicándose en la siguiente ronda.
 - **valorEfectoPendiente: int**
Guarda el valor numérico del efecto pendiente, por ejemplo 15 puntos adicionales de ataque.
 - **tieneEfectoPendiente: boolean**
Indica si existe un efecto de una Digievolución anterior que todavía debe aplicarse en la siguiente ronda.
-

Clase Ronda

- **diegimon1: Diegimon**
Guarda el Digimon seleccionado por el primer entrenador para participar en la ronda.
- **diegimon2: Diegimon**
Guarda el Digimon seleccionado por el segundo entrenador.

- **ataqueTotal1: int**
Almacena el resultado final del ataque calculado para el primer Digimon.
 - **ataqueTotal2: int**
Almacena el ataque total calculado para el segundo Digimon.
 - **ganador: int**
Guarda el resultado de la ronda. En nuestro programa:
 - 1 representa al entrenador 1.
 - 2 representa al entrenador 2.
 - 0 representa un empate.
 - **entrenador1: Entrenador**
Guarda una referencia al primer entrenador. Se utiliza, entre otras cosas, para aplicar o guardar los efectos pendientes de una Digievolución.
 - **entrenador2: Entrenador**
Guarda una referencia al segundo entrenador con la misma finalidad.
-

Clase Batalla

- **entrenador1: Entrenador**
Representa al primer participante de la batalla.
- **entrenador2: Entrenador**
Representa al segundo participante.
- **rondas: Ronda[]**
Es un arreglo utilizado para almacenar las cuatro rondas que componen la batalla.
- **CantidadRondas: int**
Lleva el control de cuántas rondas ya fueron registradas y permite saber cuándo se han completado las cuatro rondas.

4. Propósito de cada método de cada clase:

Clase Digievolucion

- **Diegievolucion(String nombre, String tipoEfecto, int valor, int probabilidad)**
Es el constructor de la clase. Recibe el nombre de la Digievolución, el tipo de efecto, el valor del efecto y su probabilidad de activación. Su función es inicializar todos los datos necesarios para crear una Digievolución y preparar el objeto Random.
 - **intentarActivar(): boolean**
Genera un número aleatorio entre 0 y 100 y lo compara con la probabilidad asignada. Devuelve true si la Digievolución se activa y false si no se activa.
 - **getNombre(): String**
Devuelve el nombre de la Digievolución.
 - **getTipoEfecto(): String**
Devuelve el tipo de efecto asociado a la Digievolución, por ejemplo ATAQUE, DEFENSA o DANO.
 - **getValor(): int**
Devuelve el valor numérico del efecto de la Digievolución.
 - **getProbabilidad(): int**
Devuelve la probabilidad de activación de la Digievolución.
-

Clase Digimon

- **Digimon(String nombre, String tipo, int ataque, int defensa, Digievolucion digievolucion)**
Es el constructor de la clase. Inicializa el nombre, tipo, ataque, defensa y Digievolución del Digimon. También establece que inicialmente no ha sido usado y que todos sus bonos comienzan en cero.
- **calcularEfectoTipo(Digimon rival): int**
Compara el tipo del Digimon con el tipo de su rival. Devuelve 20 si existe ventaja, -10 si existe desventaja y 0 cuando el enfrentamiento es neutral.
- **activarDigievolucion(): boolean**
Intenta activar la Digievolución utilizando su probabilidad. Si tiene éxito, aplica el efecto correspondiente sobre ataque, defensa o daño al rival. Devuelve true cuando se activa y false cuando falla.
- **aplicarEfectoTemporal(String tipoEfecto, int valor): void**
Recibe un tipo de efecto y un valor, y aplica temporalmente ese efecto al Digimon. Este método permite aplicar en la nueva ronda un efecto de Digievolución que quedó pendiente de la ronda anterior.

- **limpiarEfectosTemporales(): void**
Reinicia a cero los bonos temporales de ataque, defensa y daño directo después de utilizarlos en una ronda.
- **calcularAtaqueTotal(Diegimon rival): int**
Calcula el resultado final del ataque del Digimon. Utiliza su ataque base, los bonos de ataque, el efecto de tipo, la defensa del rival, los bonos defensivos del rival y cualquier daño especial. Devuelve el ataque total obtenido.
- **marcarComoUsado(): void**
Cambia el estado del Digimon para indicar que ya participó en una ronda.
- **estaDisponible(): boolean**
Comprueba si el Digimon todavía puede ser seleccionado. Devuelve true si no ha sido utilizado y false si ya participó.
- **getNombre(): String**
Devuelve el nombre del Digimon.
- **getTipo(): String**
Devuelve el tipo del Digimon.
- **getAtaque(): int**
Devuelve su ataque base.
- **getDefensa(): int**
Devuelve su defensa base.
- **getDigievolucion(): Diegievolucion**
Devuelve el objeto Diegievolucion asociado al Digimon.
- **getBonusAtaque(): int**
Devuelve el bono temporal de ataque.
- **getBonusDefensa(): int**
Devuelve el bono temporal de defensa.
- **getDanioAlRival(): int**
Devuelve el valor de daño directo que afecta al rival.

Clase Entrenador

- **Entrenador(String nombre)**
Constructor que crea un entrenador, inicializa un arreglo con espacio para cuatro Digimon, establece sus rondas ganadas en cero y deja inicialmente vacío cualquier efecto pendiente.
- **guardarEfectoSiguiente(Diegievolucion digievolucion): void**
Guarda el tipo y valor de una Digievolución que se activó para que su efecto pueda utilizarse también en la siguiente ronda.

- **aplicarEfectoPendiente(Diegimon digimon): void**
Comprueba si el entrenador tiene un efecto pendiente. Si existe, lo aplica al Digimon seleccionado para la nueva ronda y después elimina el efecto pendiente para evitar aplicarlo más veces.
- **agregarDiegimon(Diegimon digimon): boolean**
Agrega un Digimon al equipo siempre que todavía haya espacio y no exista otro Digimon con el mismo nombre. Devuelve true si pudo agregarse y false si no fue posible.
- **getDiegimon(int posicion): Diegimon**
Recibe una posición del arreglo y devuelve el Digimon almacenado en ella. Si la posición es inválida, devuelve null.
- **seleccionarDiegimon(int posicion): Diegimon**
Permite seleccionar un Digimon mediante su posición. Comprueba que la posición sea válida y que el Digimon no haya sido utilizado. Si la selección es correcta, lo marca como usado y lo devuelve; de lo contrario devuelve null.
- **sumarRondaGanada(): void**
Incrementa en uno el contador de rondas ganadas del entrenador.
- **getNombre(): String**
Devuelve el nombre del entrenador.
- **getCantidadActual(): int**
Devuelve la cantidad de Digimon agregados al equipo.
- **getRondasGanadas(): int**
Devuelve el número de rondas ganadas por el entrenador.

Clase Ronda

- **Ronda(Diegimon diegimon1, Diegimon diegimon2, Entrenador entrenador1, Entrenador entrenador2)**
Constructor que recibe los dos Digimon y los dos entrenadores que participan en la ronda. Inicializa los ataques totales y el ganador en cero.
- **ejecutarRonda(int accion1, int accion2): int**
Es el método principal de la ronda. Aplica primero los efectos pendientes, procesa si cada entrenador decidió utilizar Digievolución, calcula los ataques totales, compara los resultados y establece quién ganó. Finalmente limpia los efectos temporales. Devuelve 1 si gana el primer entrenador, 2 si gana el segundo y 0 si existe empate.
- **getAtaqueTotal1(): int**
Devuelve el ataque total obtenido por el primer Digimon.

- **getAtaqueTotal2(): int**
Devuelve el ataque total obtenido por el segundo Digimon.
 - **getDiegimon1(): Diegimon**
Devuelve el primer Digimon que participa en la ronda.
 - **getDiegimon2(): Diegimon**
Devuelve el segundo Digimon.
 - **getGanador(): int**
Devuelve el valor utilizado para identificar al ganador de la ronda.
-

Clase Batalla

- **Batalla(Entrenador entrenador1, Entrenador entrenador2)**
Constructor que recibe los dos entrenadores participantes, crea el arreglo que almacenará las cuatro rondas e inicializa el contador de rondas en cero.
 - **registrarRonda(Ronda ronda): boolean**
Guarda una ronda dentro de la batalla siempre que no se hayan completado las cuatro. También consulta quién ganó y aumenta las rondas ganadas del entrenador correspondiente. Devuelve true si pudo registrar la ronda y false cuando ya existen cuatro.
 - **batallaTerminada(): boolean**
Comprueba si ya se registraron las cuatro rondas. Devuelve true cuando la batalla está completa y false cuando aún faltan rondas.
 - **obtenerGanadorBatalla(): int**
Compara las rondas ganadas por ambos entrenadores. Devuelve 1 si ganó el primer entrenador, 2 si ganó el segundo y 0 si ambos consiguieron la misma cantidad de victorias.
 - **getEntrenador1(): Entrenador**
Devuelve el primer entrenador.
 - **getEntrenador2(): Entrenador**
Devuelve el segundo entrenador.
 - **getCantidadRondas(): int**
Devuelve cuántas rondas han sido registradas en la batalla.
-

Clase Main

- **main(String[] args): void**
Es el método principal y punto de inicio del programa. Crea el Scanner, solicita los nombres de los jugadores, crea entrenadores, Digievoluciones y Digimon, agrega los equipos, crea la batalla y controla las cuatro rondas. También

solicita las elecciones de los jugadores, muestra los resultados de cada ronda y presenta el ganador final.